



Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

A third generation genetic algorithm NSGAIII for task scheduling in cloud computing

Latreche Imene^{a,*}, Slatnia Sihem^a, Kazar Okba^{a,b}, Batouche Mohamed^c^a Computer Sciences Department, University of Biskra, BP 145, 07000, Biskra, Algeria^b Department of Information Systems and Security, College of Information Technology, United Arab Emirates University, United Arab Emirates^c Department of Information Technology, College of Computer and Information Sciences, Princess Nourah bint Abdulrahman University, P. O. Box 84428, Riyadh 11671, Saudi Arabia

ARTICLE INFO

Article history:

Received 12 October 2021

Revised 10 March 2022

Accepted 15 March 2022

Available online 31 March 2022

Keywords:

Metaheuristic

Evolutionary Many-Objective Algorithm

NSGAIII

Tasks scheduling

Cloud Optimization

ABSTRACT

Task scheduling in the cloud is perceived as a difficult Multi-objective optimization problem. It refers to the assignment of user tasks on the available cloud virtual machines. This problem can be solved effectively by combining two or more approaches for improving task execution and increasing the use of resources. In this article, a third-generation Multi-objective optimization method called Non-dominated Sorting Genetic Algorithm (NSGA-III) was used for the first time to our knowledge to schedule a set of user tasks on a set of available virtual machines (VMs) in the cloud based on a new Multi-objective adaptation function to minimize the runtime (TE), the power consumption (CE), and the cost (cout). Furthermore, the performance of NSGAIII was compared with the performance of its previous version, Non-dominated Sorting Genetic Algorithm (NSGAII) where NSGAIII results outperform the results of NSGAII. The experimental results of the proposed method are encouraging, as they are used to show their effectiveness in solving such problems.

© 2022 The Authors. Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

Cloud computing can be defined as an economic and technological revolution where Information Technologies (IT) resources (CPU, Memory, Storage, etc.) are provided as a service via a secure Internet connection. The various cloud features such as on-demand, self-service, and quality of service, usage-based billing, virtualization, and elasticity make the cloud popular in industries and research communities. Beside, Cloud Computing presents a better solution for managing data and infrastructure (Slaheddine, 2012; Thibaud, 2012).

With the development of IT services and resources for Cloud computing (see (Franck, 2007) and (Nassima and Salima, 2016)), many challenges have emerged; for example, security and reliability, as for resource management problems we have “the tasks scheduling”. It can be considered as the most important part in

the cloud that is used to distribute specific tasks to certain resources at a given time to optimize a set of Quality of Service (QoS) values for efficient use of its resources.

The latter is perceived as a combinatorial optimization problem, where it is impossible to find the global optimal solution using simple algorithms or rules First in First Out (FIFO) and Earliest Deadline First (EDF) (Thibaud, 2012). It is well known as an NP-complete problem because it has to schedule a large number of tasks within the available resources.

Scheduling tasks (Sonia, 2014) in the cloud is considered as a complex Multi-objective optimization problem that increases with the augmentation of parameters number because there are several performance parameters to be taken into consideration (run-time, cost, and energy..).

Metaheuristics of Pareto approaches such as evolutionary algorithms are good candidates for solving cloud tasks scheduling problem. As they directly use the concept of Pareto optimality in their process of research. The selecting process of the generated solutions is based on the concept of non-dominance (Elmir, 2017).

In this work we will use a metaheuristic method of 3rd generation Multi-objective optimization “the non-dominated genetic sorting algorithm (NSGA-III)” (NSGA-II with multiple objectives based on reference points).

The main contributions of the proposed work are:

* Corresponding author.

E-mail address: imene.latreche@univ-biskra.dz (I. Imene).

Peer review under responsibility of King Saud University.



Production and hosting by Elsevier

- 1) The presentation of the NSGA-III algorithm and its implementation to solve the problem of task scheduling on cloud computing.
- 2) As the NSGA-III algorithm is an improvement of the NSGA-II, we highlighted the limitations of the previous version NSGA-II as well as the modifications done to solve them.
- 3) The presentation and explanation of the Normalization Process Problem that has been found during the implementation of the algorithm. Note that this detail was not mentioned in the original paper of the algorithm.
- 4) The presentation of the appropriate solution to the normalization process problem after applying various techniques.
- 5) Compare the results of the proposed method with its early version, non-dominated genetic sorting algorithm (NSGA-II).

The rest of the paper is organized as follows. Section 2 presents the related works. Section 3 presents a background on the bases notions. Section 4 explains the proposed improved optimization system for cloud computing, section 5 presents the Data structures used. However, experimental results and discussion are presented in Section 6. Moreover, Section 7 presents the Discussion and the Comparative Study. Finally the conclusion and the future perspectives are drawn in Section 8.

2. Related works

Metaheuristics such as evolutionary algorithms are good candidates to solve the problem of tasks scheduling in cloud computing which is a difficult problem. Among the methods of scheduling in the cloud, there are the classic methods such as Earliest Deadline First (Franck, 2007); Round Robin, Shortest Job First (Insaf, 2016); and the following intelligent methods.

Atul Vikas and Dharmendra (Atul and Dharmendra, 2015) proposed a Multi-objective Tasks Scheduling Algorithm for Cloud Computing Throughput Optimization. Furthermore, the objective of this work is to assign a set of user tasks to a set of VMs to minimize the runtime (optimal runtime). The broker receives a task list and a list of VMs. Assign each QoS value to define its priority value and each VM a QoS value. Sort the tasks by dominance in which the process of linking or allocating VMs to a task is done sequentially according to the list of tasks and VMs. The first virtual machine in the list of VMs to the first task in the task list and the second virtual machine in the list of VMs with the second task in the task list. Once the allocation has reached the last virtual machine, the next task will be resubmitted to the first virtual machine in the list of VMs and the allocation process will be repeated for all tasks. Thus, the results of the proposed algorithm were compared with traditional algorithms. In order to improve its powerful more objectives (QoS) may be taken into consideration.

Panda and Prasanta (Panda and Prasanta, 2015) propose an Allocation-Aware Task Scheduling for Heterogeneous Multi-Cloud Systems, the objective is to assign a set of client tasks to the cloud VMs so that the runtime is minimized using the ATS algorithm (Allocation-Aware Task Scheduling). In the cloud model, a multi-cloud system in which each cloud has its own data center made up of a set of servers to deploy virtual machines. Each Cloud Service Provider (CSP) has its scheduler which assigns tasks to the VMs using its own scheduling strategy. However, the cloud scheduling strategy can be different.

Jena (Jena, 2015) Propose a Multi-objective Task Scheduling in Cloud Environment Using Nested Particle Swarm Optimization (PSO) Framework. The objective of this work is to optimize the energy and the processing time using a Multi-objective optimization method nested Particle Swarm optimization (TSPSO). Therefore, the results of TSPSO were compared with best resource

selection (BRS) and Random Scheduling Algorithm (RSA) where the proposed method outperforms the BRS and RSA. Thus, the optimization model should add more objectives and should focus more robust algorithm.

Shekhar and Mala (Shekhar, 2014) Proposed task scheduling optimization of independent tasks in cloud computing using an enhanced genetic algorithm. The objective of this work is to minimize runtime and balance the load of the resources. Generally, in the Genetic Algorithm (GA), the initial population is generated randomly; these randomly generated solutions are not efficient because there is a probability that a solution can be repeated. Therefore, the idea of the algorithm proposed in this work is to generate the initial population by the Enhanced Max-Min algorithm. Therefore, the future perspective of this work is using the execution cost of the resource as a fitness criterion.

Zuo et al. (Zuo et al., 2015) presented an improved Multi-objective Ant-Colony Optimization Algorithm (PBACO) to solve the Multi-objective Task Scheduling problem by optimizing four objectives: the cost, makespan, resource utilization, and time deadline. At first, a resource cost model that defines the demand on tasks on resources with more details was proposed toward the biodiversity of resources and tasks in cloud computing. The proposed PBACO was compared with the original ant-colony algorithm. Moreover, the classical heuristic algorithm Min-Min algorithm and First Come First Serve (FCFS) scheduling. Thus, the obtained results show that the proposed method is better than other similar methods. However, the method still fell into the local optimum and the major comparative study was with traditional methods.

Naik et al. (Naik et al., 2019) have presented a Multi-objective optimization hybrid algorithm Non-dominated Sorting Genetic Algorithm-II (NSGA-II) and Gravitational Search Algorithm (GSA), titled NSGA-II & GSA. This work aims to choose the most appropriate virtual machines to schedule tasks on cloud data centers by optimizing three objectives: response time, cost, energy consumption. The GSA selects the VMs which have less energy consumption which will be the population of NSGA II, where the latter will find the VMs with less response time and cost. Therefore, the results of the proposed algorithm outperform the obtained results of the original NSGA II and the Non-dominated ranked genetic algorithm (NRGA).

In (Adamuthe et al., 2013), Amol et al. have implemented three algorithms that are genetic algorithm, Non-dominated Sorting Genetic Algorithm and, Non-dominated Sorting Genetic Algorithm-II to solve virtual machines placement problems in cloud computing based on three objectives which are maximizing profit, maximizing load balancing, and minimizing recourse wastage. Thus, the obtained results of the implemented algorithm were good solutions. However, NSGA-II gives better and diversified solutions compared to the two others.

In (Zhou et al., 2020), Zhou et al. have proposed a novel Multi-objective algorithm to schedule tasks on a cloud environment named MGGS (modified genetic algorithm (GA) combined with greedy strategy). The MGGS algorithm was evaluated based on total completion time, average response time, and QoS parameters. The proposed genetic algorithm outperforms other task scheduling algorithms such as the classical GA, First Come First Serve (FCFS), and Min-Min algorithms.

Alkayal et al. (Alkayal et al., 2016) have used a Multi-objective particle swarm optimization algorithm (MOPSO) based on a new ranking strategy for cloud scheduling. The main aim is to optimize three conflict objectives which are the expected completion time (ECT), task execution cost (TEC), and VM processing. Therefore, the improvements are shown in terms of waiting time and throughput. However, the impact of resource utilization and load balancing is missing.

In (Yuan et al., 2015), the authors have introduced a Non-dominated Sorting Genetic Algorithm based on Grid Partitioning. They aim to improve the diversity and the convergence of the existing NSGAs algorithms, to reduce the computation time. Besides, the introduced algorithm called NSGA-G has proved its efficiency by outperforming the other algorithms such as NSGA versions (NSGA II/NSGA III) and Multi-objective Evolutionary Algorithm Based on Decomposition (MOEA/D). However, the algorithm was evaluated based on DTLZ problems which, are often used for estimating the quality of Multi-objective Evolutionary Algorithms (Table 1).

In (Slim and Rituparna, 2016), Deb and Jain. Proposed the NSGA-III algorithm that remains similar to the NSGA-II algorithm with some changes in its selection mechanism. As it focuses on members who are not dominated by the population but who are close to a set of reference points provided. NSGA-III is applied to several multi-purpose problems with two to 15 objectives (See (Kalyanmoy and Fellow, 2013) and (Haitham and Kalyanmoy, 2017)).

3. Background

3.1. Optimization problem

An optimization problem is defined by:

- A research (decision) space: set of solutions or configurations made up of the different values taken by the decision variables.
- One or more function (s) called objective (s), to be optimized (minimize or maximize).
- A set of constraints to be observed.

The optimal solution of the problem is to find out the point or a set of points in the search space that best satisfies the objective function. The result is called **the optimal or optimum value** (Mahdi, 2007).

Optimization problems can be divided into two classes; when a solution is associated with a single value, we speak about single-objective problems, when it is associated with several values; we speak about Multi-objective (or multi-criteria) problems (Layeb, 2010).

3.2. Mono-objective optimization

When only one objective (criterion) is given, the optimization problem is mono-objective. In this case, the optimal solution is

clearly defined; it is the one that has the optimal cost (minimum, maximum) (Mahdi, 2007).

3.3. Multi-objective optimization

The real problems are not always so “simple”. Sometimes we need to define several objectives to be optimized, for the same problem.

Multi-objective problems have the particularity of being more difficult to deal with than their single-objective counterparts as difficulty lies in the lack of a total order relationship between the solutions. In this case, the optimal or good quality solution is no longer a single one, but a set of solutions compromised between the different objectives to be optimized. Therefore, the most famous and the most total order relationship used is the **dominance relation** in the Pareto sense. The set of best compromises is called the **Pareto Front**, the **Compromise Surface**, or the Set of Effective Solutions (Mahdi, 2007).

Definition 1. Multi-objective optimization problem.

An optimization problem with multiple objectives can be represented by the following program:

Optimize $F(S) = (f_1(S), f_2(S) \dots f_p(S))$.

While $S \in \Omega$ and $p \geq 2$.

S: a solution vector $(x_1 \dots x_n)$ of a space of dimension n , representing instances of the decision variables x_i .

Ω : represents the set of feasible solutions respecting a set of constraints C of equality, inequality and explicit bounds.

$F(f_1, f_2 \dots f_p)$: is the objective function vector to be optimized, and p represents the number of objectives.

Definition 2. Dominance.

y domine z if and only if $\forall i \in [1..n] y_i \leq z_i$ and $\exists i \in [1..n]/y_i < z_i$ (Talbi, 2000).

Definition 3. Pareto optimality.

An $x^* \in C$ solution is Pareto optimal if and only if there is no $x \in C$ solution such that $F(x)$ dominates $F(x^*)$. (see Fig. 1)

Definition 4. The compromise surface.

The solutions obtained by ranking based on the Pareto dominance principle form what is called the compromise surface (or Pareto Front).

Table 1
Summarizing the aforementioned related works.

Articles	Optimization methods	Objectives	Limitations
R K Jena (Jena, 2015)	nested Particle Swarm optimization (TSPSO)	Optimize the energy and the processing time	-Comparison with classical algorithms.
Shekhar and Mala (Shekhar, 2014)	enhanced Genetic algorithm	Minimize runtime and balance the load of resources	-More objectives must be added.
Zuo et al (Zuo et al., 2015)	an improved Multi-objective Ant Colony optimization Algorithm (PBACO)	The cost, makespan, resource utilization, and time deadline.	-More objectives must be added to evaluate the performance.
Naik et al (Naik et al., 2019)	NSGA-II & GSA	Response time, cost, energy consumption.	-The method still falling into the local optimum. - Comparative study was with traditional methods.
Amol et al (Adamuthe et al., 2013)	NSGA II	Maximizing profit, maximizing load balancing and minimizing recourse wastage.	Faster convergence that could lead to premature convergence.
Zhou et al (Zhou et al., 2020)	MGGS	Total completion time, average response time, and QoS parameters	Diversity cannot
Alkayal et al (Alkayal et al., 2016)	MOPSO based on a new ranking strategy.	expected completion time (ECT), task execution cost (TEC) and VM processing	be preserved with more than two objectives.
			Comparative study was with traditional methods.
			The impact of resource utilization and load balancing are missing.

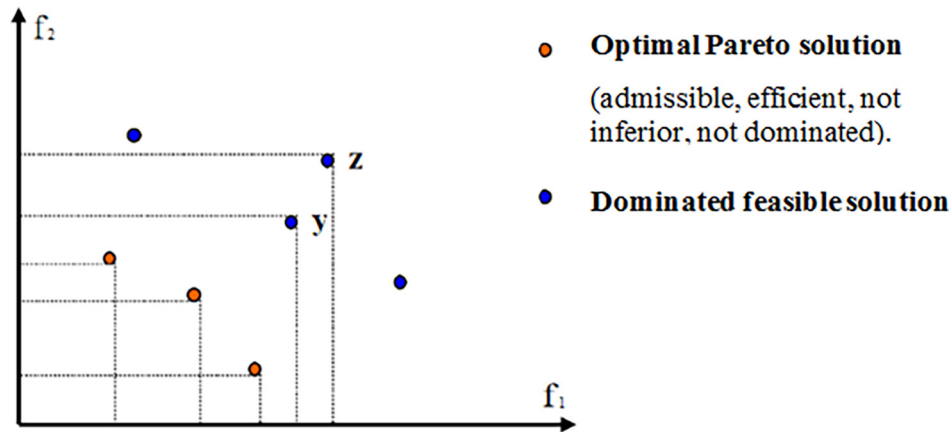


Fig. 1. Exact set of optimal Pareto solutions (Talbi, 2000).

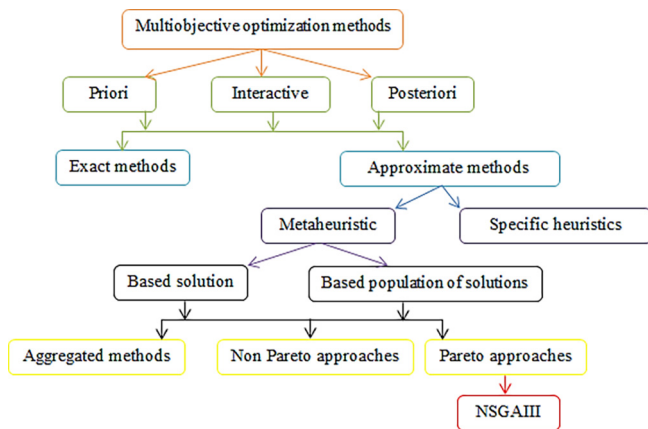


Fig. 2. Classification of Multi-objective optimization methods (Elmir, 2017).

3.4. Multi-objective optimization methods

Fig. 2 shows the different Multi-objective classification methods, for more details visit (Elmir, 2017).

In our work, we are interested in the approximate methods especially, Metaheuristics ones that are often inspired by optimization mechanisms encountered in nature. They provide general solving schemes allowing them to be potentially applied to all problems. Several classifications of Meta-heuristics have been proposed. Thus, the most distinguished are two categories: those based on a single solution and those based on a population of solutions (Mahdi, 2007).

Specifically, we are interested in genetic algorithms of the Pareto approaches that are Metaheuristics methods based population of solution.

3.5. NSGA-III: Non-dominated Sorting genetic algorithm III

Deb and Jain (Slim and Rituparna, 2016) proposed NSGA-III which remains similar to the NSGA-II algorithm with some changes in its selection mechanism.

As the NSGAIII algorithm is an improvement of NSGAII, we first present a brief description of the original NSGA-II algorithm.

3.5.1. NSGAII algorithm

A random parental population P_t with N individuals is generated. The following steps are repeated until the termination criterion is satisfied. First, the algorithm begins by creating an offspring population Q_t with N individuals obtained by applying genetic

operators to P_t . Subsequently, P_t and Q_t are merged to form a new population R_t of size $2N$ individuals. As these individuals will be sorted according to Pareto dominance in Different fronts ($F_1, F_2, \dots, F_n$). Then, a new population S_t is constructed from selecting the fronts starting from the first front F_1 until the size of the population S_t becomes equal to N or for the first time greater than N . Suppose that the last accepted level is the l -th level. Therefore, all solutions of the level $(l+1)$ are rejected. In most cases, the last front F_l is partially accepted. In such a case, only the solutions which maximize the diversity of the l -th front are chosen. This is achieved by a computationally efficient but approximate niche-preservation operator that computes the crowding distance for each last-level member as the sum of objective-wise normalized distance between two neighboring solutions. Therefore, the most dispersed solutions (having larger crowding distance) are added until the population P_{t+1} is completed (Kalyanmoy and Fellow, 2013).

3.5.1.1. NSGA-III work process. As indicated before, NSGA-III is similar to NSGAII with some changes in its selection mechanism (from 13 to 18) in Fig. 3; the selection mechanism of NSGAII is a crowding distance measure that does not perform well for many-objective problems. Thus, the selection mechanism in NSGA-III is modified by conducting a more systematic analysis of members in S_t by respecting the supplied reference points (Yuan et al., 2014).

NSGA-III uses a predefined set of reference points to ensure diversity in obtained solutions. However, the chosen reference points can either be predefined in a structured manner or supplied preferentially by the user (to get more details, see (Kalyanmoy and Fellow, 2013)).

The selection mechanism of NSGAIII works as follow:

It starts by normalizing the population members and the given reference points so that all objective vectors and reference points have proportional values. Next, the perpendicular distance (d) between a solution in S_t and each of the reference lines that join the ideal point with the reference points will be calculated. Thus, each individual of S_t is associated with the reference point having the minimum perpendicular distance. Next, a conservative niche strategy is used to choose the FL members that are associated with the least represented benchmarks in P_{t+1} (Slim and Rituparna, 2016). The process is detailed in section four.

Fig. 3 explain how the NSGAIII algorithm works.

4. Proposed approach

In this research, we will use a third-generation Multi-objective metaheuristic optimization method, which is the Non-dominated

Algorithm 1 Generation t of NSGA-III procedure**Input:** H structured reference points Z^s or supplied aspiration points Z^a , parent population P_t **Output:** P_{t+1}

```

1:  $S_t = \emptyset, i = 1$ 
2:  $Q_t = \text{Recombination+Mutation}(P_t)$ 
3:  $R_t = P_t \cup Q_t$ 
4:  $(F_1, F_2, \dots) = \text{Non-dominated-sort}(R_t)$ 
5: repeat
6:    $S_t = S_t \cup F_i$  and  $i = i + 1$ 
7: until  $|S_t| \geq N$ 
8: Last front to be included:  $F_l = F_i$ 
9: if  $|S_t| = N$  then
10:   $P_{t+1} = S_t$ , break
11: else
12:   $P_{t+1} = \bigcup_{j=1}^{l-1} F_j$ 
13:  Points to be chosen from  $F_l$ :  $K = N - |P_{t+1}|$ 
14:  Normalize objectives and create reference set  $Z^r$ :  $\text{Normalize}(f^n, S_t, Z^r, Z^s, Z^a)$ 
15:  Associate each member  $s$  of  $S_t$  with a reference point:  $[\pi(s), d(s)] = \text{Associate}(S_t, Z^r)$ 
    %  $\pi(s)$ : closest reference point,  $d$ : distance between  $s$  and  $\pi(s)$ 
16:  Compute niche count of reference point  $j \in Z^r$ :  $\rho_j = \sum_{s \in S_t/F_l} ((\pi(s) = j) ? 1 : 0)$ 
17:  Choose  $K$  members one at a time from  $F_l$  to construct  $P_{t+1}$ :
    Niching ( $K, \rho_j, \pi, d, Z^r, F_l, P_{t+1}$ )
18: end if

```

Fig. 3. The work process of NSGAIII algorithm (Haitham and Kalyanmoy, 2017).

Sorting Genetic Algorithm (NSGA-III), NSGA-II with multiple objectives based on reference points. However, the reasons for our choice are: the NSGA-III has demonstrated very good results on several problems that involve from three to 15 objectives. Furthermore, it is well known for its ability to find well-converged and well-diversified solutions, as it does not require any additional parameter (Slim and Rituparna, 2016; Haitham and Kalyanmoy, 2017; Ibrahim et al., 2016), as well as it was not used to the cloud tasks scheduling to our knowledge.

The objective of our work is to schedule a set of user tasks on a set of available VMs in the cloud to minimize the runtime, energy consumption, and cost.

The Fig. 4 presents the architecture of the proposed system.

1. Cloud clients: Clients will send a set of requests (tasks) to the cloud service provider to request services over the internet.

2. Broker: represents an intermediary entity between customers and cloud providers; this is where the scheduling takes place.

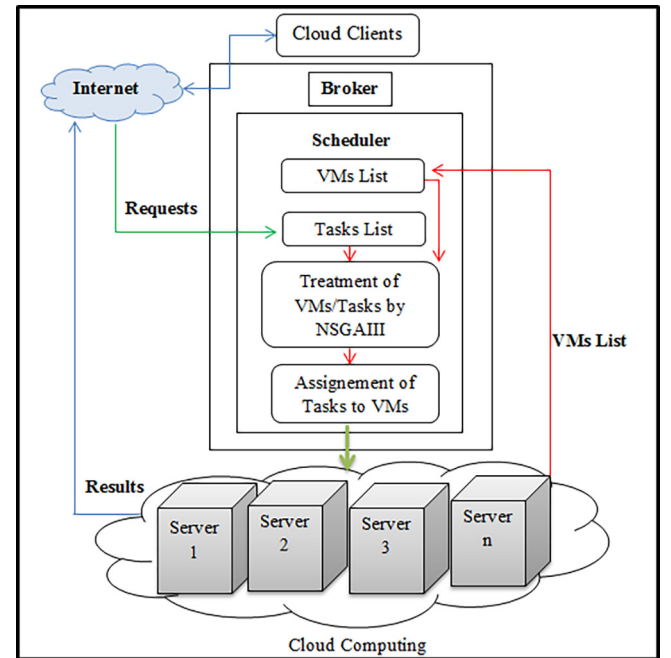
3. Tasks list: represents all the requests sent by customers to the cloud service provider.

4. VMs list: represents all the virtual machines available in cloud servers.

5. Treatment of VMs/Tasks by NSGAIII:

6. Assignment of tasks to VMs: This is the last step where the broker will apply the optimal solution and assign each task to a specific VM.

7. Data centers: Are centers at which the execution of the tasks takes place, once completed the results will be sent directly to the client.

**Fig. 4.** System Architecture.

4.1. NSGA-III algorithm using multiples features selection

A new improved NSGA-III using multiples features selection for task allocation in cloud computing (see Fig. 5) is based on two main phases.

4.1.1. Modeling phase

1. User request is defined by the set of tasks assigned to each VM. Thus, it is represented by a list of the list or the first represents the VMs and the second represents the cloudlets. Besides, the Gene in the individual (solution) will represent the tasks associated with VM (see Fig. 6).

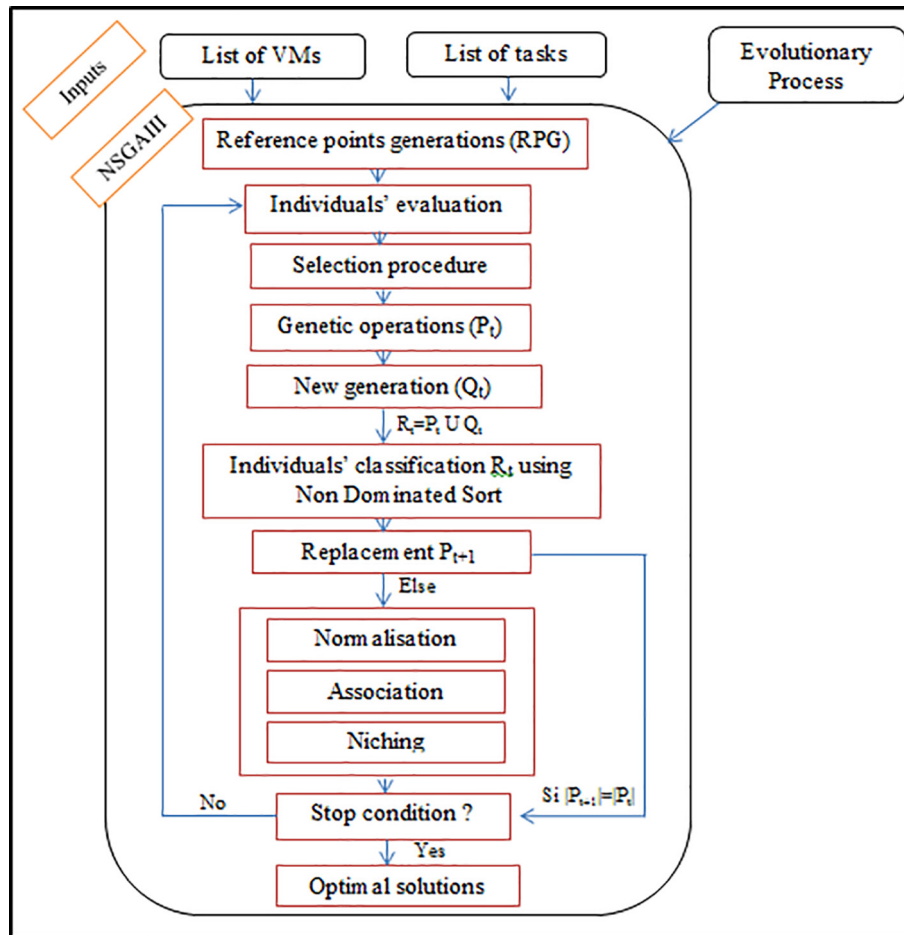


Fig. 5. The NSGAIII algorithm for the selection of tasks using reference points.

- Population is a set of solutions (individuals) P_t represented by a list of size N . The individuals of the initial population will be generated randomly. While, the size N is chosen by the user (see Fig. 7).
- Reference points after the initialization of the population, we will define all the reference points which will ensure the diversity of the obtained solutions.

The number of these points is calculated from the ones of objectives M and the number of divisions P . The chosen reference points can be predefined in a structured manner or preferentially pro-

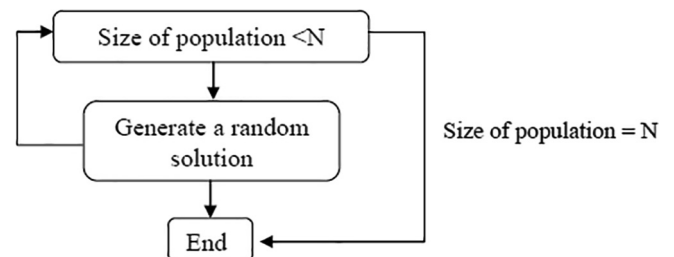


Fig. 7. Organization chart of the initial population.

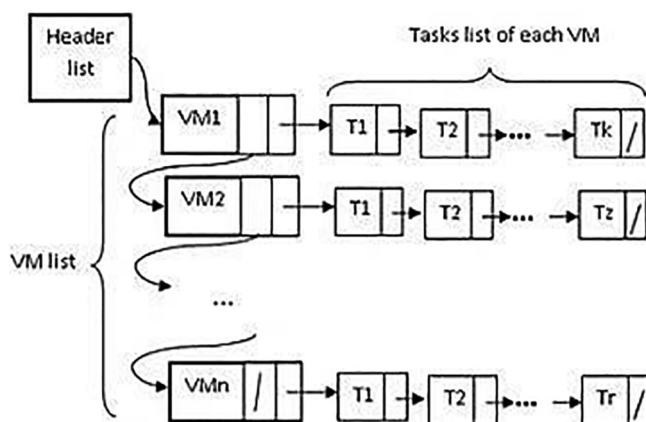


Fig. 6. Representation of an individual.

vided by the user. If they are supplied by the user, they will be normalized and then distributed in a hyperplane.

In our study, we worked with 10 reference points for a number of divisions ($p = 3$) and a number of objectives ($M = 3$), these points are created on a normalized hyper-plane (a triangle) of vertex (1.0, 0), (0.1, 0), (0.0, 1).

4.1.2. Evolution phase

In this subsection, we present the main stages of the evolution phase.

1. Evaluation of individuals

The evaluation of an individual is done by a function called objective or fitness function, the role of this function is to assign to each individual an adaptation value will have the fitness value from the objectives imposed in our system (minimize runtime, minimize energy consumption and minimize cost).

- Runtime: is the time necessary for the run of a task.
- Cost: is the total price for performing tasks on the VMs.
- Energy consumption: is the energy consumed by the data center when running a solution.

2. Selection

In this step we will choose the $N/2$ individuals, who will be the parents of the new population, we will use the uniform selection which consists of choosing individuals randomly without the intervention of the fitness value.

3. Crossing is the operator who ensures the notion of diversification of individuals, which will be done as follows:

- Select two individuals.
- Choose a cut point on the representation of the two individuals.
- Generate two children by crossing the two parts which are located just after the parents crossing point.

4. Mutation

We choose two VMs randomly and by a probability P_m in a selected individual and we swap the list of tasks for these two VMs.

5. Classification of individuals

In Multi-objective optimization, after the evaluation, we must classify the individuals according to their dominance with the objective function then establish the Pareto fronts ($F_1, F_2 \dots F_n$) (see Fig. 8).

6. Replacement

After the selection and application of the genetic operators on P_t , we obtain a new population Q_t , which will be merged with the old P_t to form R_t (which is a population of size $2N$), the latter will be evaluated and classified in a set of fronts, besides, each of these fronts is added to the population S_t up to $|S_t| \geq N$.

If $S_t = |N|$ we make $P_{t+1} = S_t$. Otherwise, the NSGAIII selection mechanism is applied using the predefined reference points to construct P_{t+1} (see Fig. 9).

6.1. Normalization

In this phase the objectives of each individual are normalized between 0 and 1, therefore the individuals will be distributed in a hyperplane of three axes (1, 0, 0) (0, 1, 0) (0, 0, 1). Fig. 10 shows the normalization process for the original NSGAIII paper (See (Kalyanmoy and Fellow, 2013)).

After a depth and precise theoretical study of the normalization process mentioned in the original paper (Kalyanmoy and Fellow, 2013), we find that it is likely to encounter problems that can influence the effectiveness of NSGAIII.

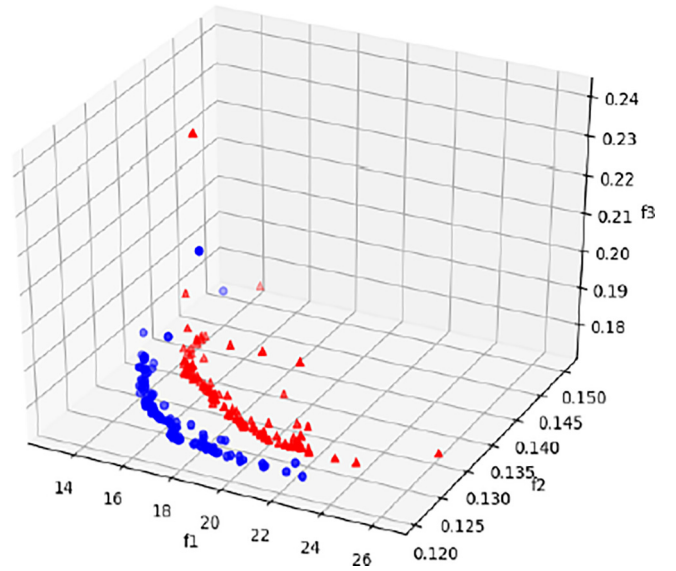


Fig. 8. Classification of individuals, Pareto Fronts.

i. The red boxes in Fig. 10 show the location of problems that may arise in the method of calculating extreme points and the method of normalizing objectives.

- The first case appears when the extreme points are duplicated and consequently influence the calculation of the intercepts (impossibility of constructing a single hyperplane).
 - The second case lies in the normalization of the objectives itself where in some cases the normalized values will exceed the value 1.
- ii. To verify the real existence of these problems, we will implement the normalization process and test it.

6.2. Association

After having normalized each objective, we must then calculate the reference lines between each reference point and the ideal point, we will then calculate the distance between each normalized individual of S_t as well as the reference lines (perpendicular distance) finally the point of reference whose line is closest to a member of the population will be associated with it (Fig. 11).

6.3. Nicheing

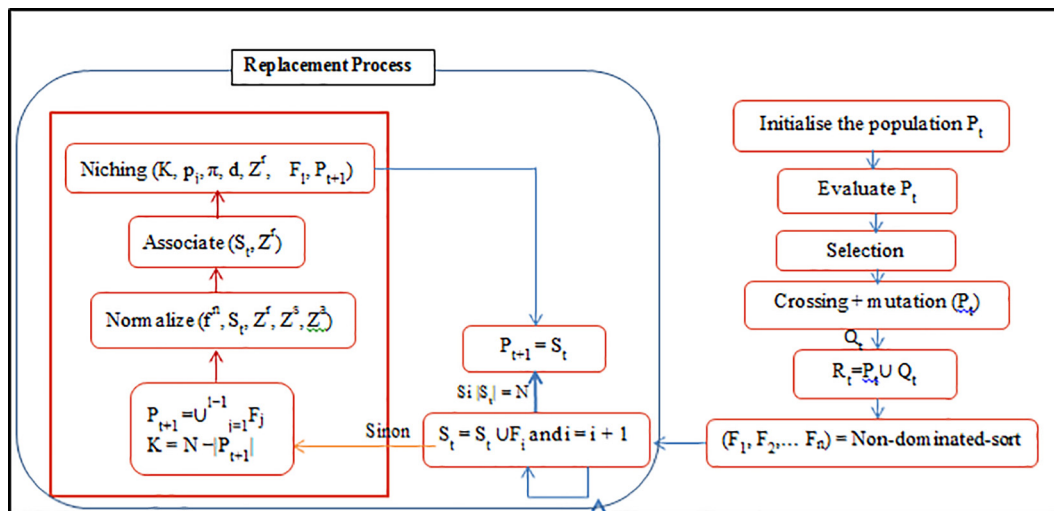


Fig. 9. Replacement process.

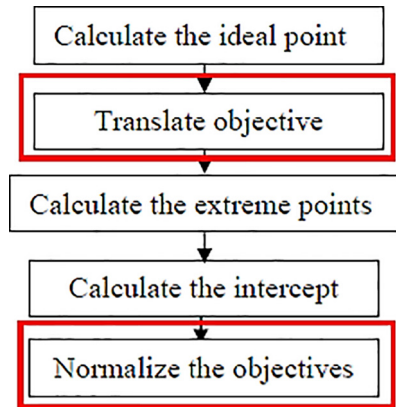


Fig. 10. Normalization process.

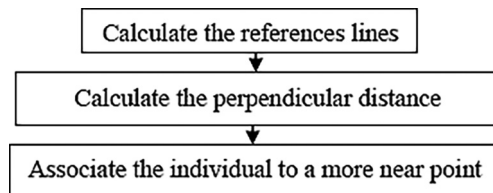


Fig. 11. Association process.

In this method, we will choose the k individuals of the last front using the results obtained by the association method.

Figure 12 shows the Niching process, the process is repeated until $P_{t+1} = P_t$.

p : number of members of the population $P_{t+1} = S_j/F_j$ associated with each reference point.

p_j : number of members of the population $P_{t+1} = S_j/F_j$ associated with the reference point j .

J_{min} : the set of reference points that has the minimum p .

$|F_j|$: the front members F_j which are associated with the reference point j .

S : $\pi(s) = j$, $s \in F_j$: member “ s ” of F_j which has the shortest distance perpendicular to the reference line.

7 Termination criteria

As long as the stop criterion which is the predefined number of generations is not satisfied, the algorithm remains in function.

The optimal solutions are the set of effective solutions located in the first front “Pareto front”, this set of solutions constitutes a

balance, in the sense that no improvement can be made on a goal without degradation of at least one another goal. When the process of the NSGAIII algorithm ends, we take the best solution obtained which is located in the last generation on the 1st Pareto front, finally, we assign each list of tasks to the corresponding VM.

4.2. The implementation of NSGAIII

4.2.1. The main algorithms

After implementing the proposed architecture (Fig. 4) in eclipse environment with Java programming language, we will present the main implemented algorithms such as Pareto front algorithm, Replacement algorithms (normalize, association, niching).

• Fast-Not-Dominated-Sort (Pareto front).

In algorithm 1, we have defined the first front (optimal solutions) and store them in a list called “NotDominatedSol”, the other solutions (dominated solutions) were stored in list called “DominatedSol”.

In algorithm 2, we have created a list of lists called “Front” to store the different Pareto fronts. First, we have added the not-dominated solutions as the first front (F_0); then we have identified the second front (members) from the dominated solutions list and added it in Front list as (F_1), and so on until the last front. However, after identifying front members, we remove them from the dominated list.

Algorithm 1 NotDominatedSolution ()

```

1: for each individual in p (pPopulation) do
2: for each individual in q (qPopulation) do
3: if q domine p then
4: Increment count by 1
5: end if
6: end for
7: if count = 0 then
8: Add p to NotDominatedSol//a list of not dominated solutions.
9: else
10: Add p to DominatedSol//a list of dominated solutions.
11: end if
12: end for
  
```

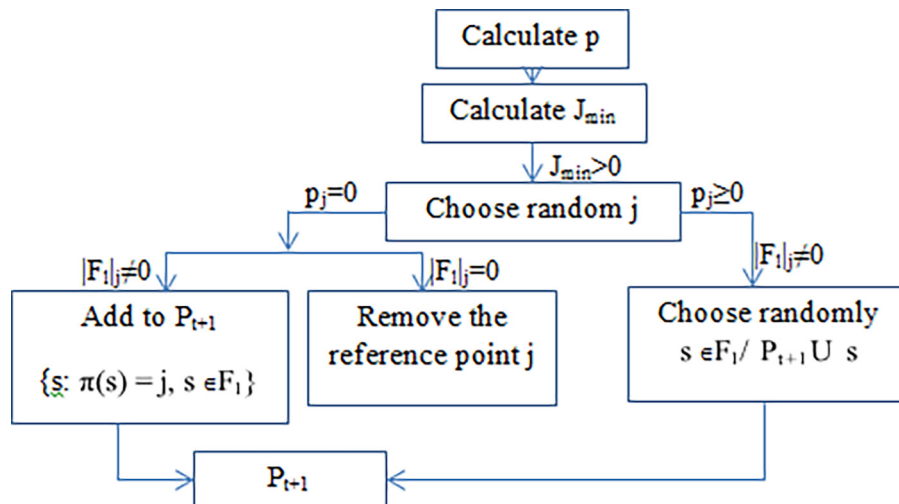


Fig. 12. Niching process.

Algorithm 2 Front (NotDominatedSol, DominatedSol)

```

1: Add NotDominatedSol to 1st Front (F0)
2:  $i \leftarrow 1$ 
3: while DominatedSol.Size ()  $\neq 0$  do
4:   for each individual in p (pPopulation) do
5:     for each individual in q (qPopulation) do
6:       if q domine p then
7:         Increment count by 1
8:       end if
9:     end for
10:    if count = 0 then
11:      Add P to Front (Fi)
12:      Add P to list
13:    end if
14:  end for
15:  if list.size ()  $\neq 0$  then
16:    Delete the individuals stored in list from the
      "DominatedSol" list
17:  end if
18:   $i \leftarrow i + 1$ 
19: end while

```

The advantage of NSGAIII lies in its ability to ensure the diversity of solutions obtained using a set of reference points and a mechanism for selecting individuals missing in the replacement phase. Therefore, a new of an improved NSGAIII is based on the three methods of normalization procedure.

- **Normalization of population members**

After the implementation of normalization, we found that the hypothesis of the existence of problems in the calculation of the extreme points and the normalized method is verified.

Algorithm 3 Normalize (Stfit)

```

1: Calculate  $Z_{\min}$  the ideal point
2: for each individual  $f_i$  ( $f_i$  Stfit) do//Stfit population with objectives
3:   for each objective m do//m number of objectives
4:      $f_{i,m} = f_i.m - Z_{\min,m}$ ;
5:   end for
6: end for
7: Calculate the extreme points
8: Calculate the intercepts
9: for each individual  $f_i$  ( $f_i$  Stfit) do
10:  for each objective m do
11:     $f_{i,m} = f_{i,m} / a_m - Z_{\min,m}$   $a_m = \text{intercepte}_m$ 
12:  end for
13: end for

```

Problem 1. As mentioned previously, cases of duplication of the extreme points can appear and cause a problem in the calculation of the intercepts (we get Nan = "not a number" as a result), hence the impossibility of building a single hyper plan.

Proposed resolution to solve this problem, Tsung-Che Chiang proposes as a solution to add a test aimed to verify the existence or absence of duplication in the calculate intercepts method in (nsga3cpp Version 1.0). This solution is detailed in the procedure below:

Algorithm 4 Cal_Intercepts (extreme_points)

```

1: Duplicate = Is_duplicate (extreme_points)//Extreme points
2: if duplicate then
3:   for each Objective m do
4:      $a_m = \text{extreme\_points}(m).m$ 
5:   //each diagonal point of the extreme points becomes an intercept  $a_m$ 
6:   end for
7: else
8:   List x = Gaussian-Elimination ()//Find the hyper plan equation
9:   //find the intercepted
10:  for each Objective m do
11:     $a_m = (1/x_m)$ ;  $a_m = \text{intercepte}_m$ 
12:  end for
13: end if

```

After the implementation of this solution, the problem of duplicate points is solved, and the intercepts are calculated.

Problem 2. The second problem encountered after solving the first is the normalization equation; here in some cases the values of the objectives exceed the value 1.

Proposed Resolution: Meticulous monitoring of the entire normalization process has allowed us to discover the cause of the problem, and which are the intercept values a_m which must be greater than the objective value f_m . More precisely the problem appears in case where the extreme points are duplicated or the intercepts compute are negative.

From all the above we have concluded that the solution proposed by Chiang, helps us to calculate the intercept without being the ideal solution. Thus, this leads us to look for another more effective method.

1. Instead of choosing the diagonal of the extreme points as intercepts a_m , we will choose the maximum values of each objective of all the extreme points.

Algorithm 5 DiagonalExtreme ()

```

1: if duplicate then
2:   for each Objective m do
3:     Max = findMax (extreme_points (0).m, extreme_points (1).m, extreme_points(2).m)
4:      $a_m = \text{Max}$ 
5:   end for
6: end if

```

After the implementation of this solution, the problem still persists, therefore, it is not the solution sought.

2. Another solution that we have experimented is to give values greater than the value 1 of the lower values.

This solution was tested only to verify the diversification of solutions distribution compared to the references points however, the obtained results are far from the aim to seek (see the Fig. 13), which led us to conclude that the problem lies in the choice of intercept values in the case of duplication.

3. A third solution is that of Yuan et al (Yuan et al., 2015) who proposes to create a FindMaxObjectives method to find the maximum values of the objectives among the current solutions. Then replace the intercept with these values in the case where the duplicated endpoints or negative intercept.

Algorithm 6 FindMaxObjectives (pop)

```

1: for each Objective m do
2:   Max_point (m) = pop (0).m
3:   for each individual i in pop do
4:     Max_point (m) = Max (Max_point (m), pop (i).m)
5:   end for
6: end for
7: return Max_point

```

Algorithm 7 Cal_Intercepts (ExtPoits)

```

1: Duplicate = Is_duplicate (extreme_points)
2: if! Duplicate then
3:   List x = gaussianElimination//Find the hyperplane equation
4:   for each objective i < 3 do//Find the intercepts
5:     interceptes.add (1/xi)
6:   if xi < 0 then
7:     negative intercept = true;
8:   break
9: end if
10: end for
11: end if
12: if Duplicate or negative intercept then
13:   max_objs = FindMaxObjectives (pop)
14:   for each objective m do
15:     Interceptes (m) = max_objs(m)
16:   end for
17: end if

```

This solution allowed us to solve the problem and make the normalization function work properly. Therefore, the problem that we mentioned above does not appear in the original paper of NSGAIII (see Fig. 14).

• Association Algorithm

Algorithm 8 AssociationOperation

```

1: for each individual pi in (pi Stfit) do
2:   Min_rp = -1;
3:   Min_dist = Double.Max_Value
4:   for each reference point ri in (ri referencepoints) do
5:     Calculate perpendicularDistance between ri and pi
6:     if perpendicularDistance < min dist then
7:       Min_dist = perpendicularDistance;
8:       Min_rp = ri;
9:     end if
10:   end for
11:   Add pi with its near ri to association list
12: end for

```

• Niching Algorithm

Algorithm 9 Niching (St, K, association, ptsize)

```

1: k 0
2: Create p
3: while k < K do
4:   Calculate Jmin;
5:   Calculate j;
6:   Calculate Ij;
7:   if Ij = 0 then
8:     if pj = 0 then
9:       Add to Pt+1 the solution of front Fi which has the minimum distance
10:    else
11:      Choose a random individual of Fi
12:    end if
13:    pj = pj + 1;
14:    Remove s of Fi
15:    k = k + 1
16:  else
17:    Remove the reference point j
18:  end if
19: end while

```

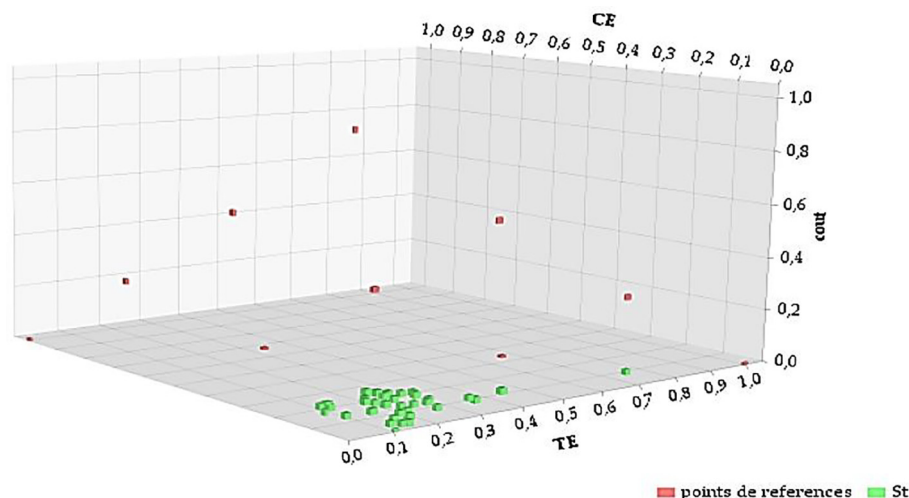


Fig. 13. Reference points, population after normalization.

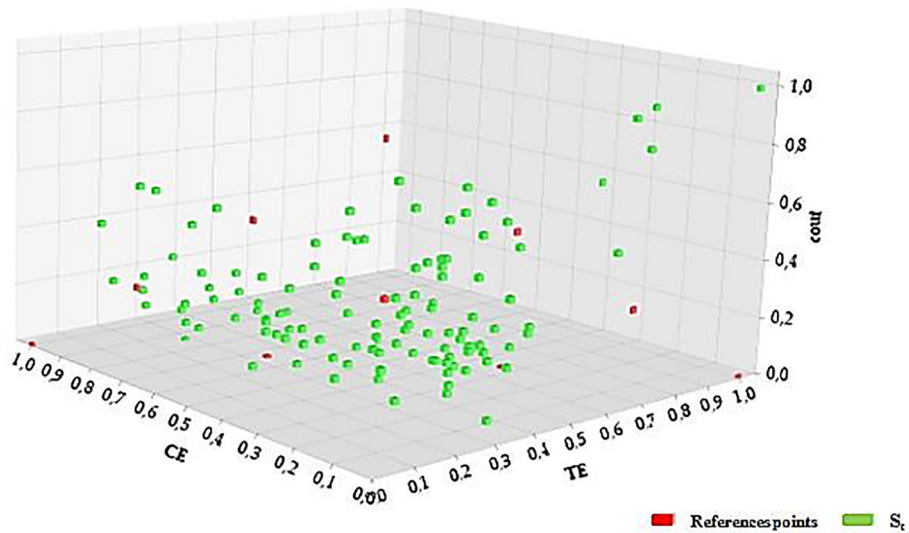


Fig. 14. Representation of reference points and the population after normalization, reference points: red and St: green.

Table 3
Experimental parameters of the algorithms.

Parameters	Values
Population size	10, 50, 100, 200, 500, 800, 1000
Iteration Number	10, 100, 500, 1000
Probability of mutation	0.01
Probability of crossing	0.9

4.2.2. Individuals assessments

Individuals evaluation is done by the fitness function, which calculates its runtime, cost and, energy consumption for each individual. In what follows we will explain the method of calculating these three values for an individual.

(a) Runtime (execution time) TE = Max (the sum of the TE of all the tasks for each VM of individual i).

Table 2
The CloudSim configurations value.

Types	Parameters	Values
Datacenter	Numbers of DC	1
	Number of Hosts in each DC	10
Hosts	The speed of Hosts	20000–200000 Mips
	Number of PEs in each host	1
VMs	The total number of VMs	30
	The speed of VMs	1000–5000 Mips
Tasks	Total number of tasks	100
	The size of the tasks	5000–15000 MI
	Number of PEs	1

(b) Cost For each VM allocated, the allocation cost is 1\$ per 3600 s.

Cost = the sum of the cost of each VM for individual i.

Table 4
The optimal solutions (1st front) of the first generation (the first three columns) and the last generation (for the rest).

Population size = 100 & number of generations = 100								
TE(s)	cost(\$)	CE(Kw)	TE(s)	cost(\$)	CE(Kw)	TE(s)	cost(\$)	CE(Kw)
32.980	0.183	0.175	24.460	0.156	0.175	18.879	0.140	0.183
21.593	0.178	0.175	15.560	0.158	0.175	16.677	0.143	0.175
20.783	0.171	0.175	15.560	0.156	0.175	16.677	0.143	0.176
18.74	0.155	0.175	16.217	0.155	0.175	16.217	0.144	0.175
22.302	0.148	0.180	15.560	0.154	0.175	18.958	0.141	0.175
30.482	0.172	0.175	18.879	0.143	0.175	15.769	0.147	0.231
19.744	0.153	0.175	16.989	0.147	0.175	15.769	0.148	0.180
20.642	0.147	0.237	16.098	0.147	0.175	16.336	0.143	0.216
19.525	0.152	0.202	18.958	0.141	0.175	18.730	0.141	0.175
17.15	0.156	0.179	16.4	0.144	0.175	16.586	0.143	0.180
19.842	0.152	0.185	15.769	0.151	0.175	16.677	0.143	0.180
17.579	0.157	0.179	16.586	0.144	0.175	16.336	0.143	0.176
20.113	0.151	0.178	18.958	0.142	0.175	18.958	0.140	0.176
19.362	0.153	0.225	18.879	0.140	0.175	16.677	0.142	0.183
20.389	0.149	0.180	15.996	0.147	0.175	15.769	0.150	0.175
19.842	0.148	0.195	15.996	0.145	0.177	15.560	0.149	0.210
24.216	0.145	0.225	16.098	0.144	0.175	16.677	0.142	0.192
24.216	0.147	0.214	16.820	0.142	0.182	16.677	0.142	0.205
19.791	0.150	0.241	18.730	0.141	0.175	15.560	0.149	0.176
18.921	0.155	0.198	16.586	0.143	0.175	15.769	0.148	0.211
20.389	0.147	0.244	18.730	0.140	0.190			
19.136	0.151	0.238	16.217	0.144	0.181			

(c) As for energy consumption we used Linear Power Model which calculates the energy as follows: $\text{PowerConsumed} = P_{\min} + (P_{\max} - P_{\min}) * \text{Utilization}$.

Where $P_{\min}$ is the minimum power consumed in the idle state and $P_{\max}$ is the maximum power consumed at the top load. The use is between 0 and 1.

5. Experimental results

Our system is developed under the Intel (R) CORE™i3 micro-portable environment (1.70 GHz CPU, 4 GB RAM) Microsoft Windows 7.

We present the experimental results of the NSGAIII algorithm used for scheduling tasks in the cloud. The experiments aim to demonstrate and evaluate the performance of the algorithm using different parameters in addition to a comparative study with another algorithm.

We will present also the CloudSim configuration used for the parameters of each of Datacenter, VMs, Hosts, cloudlets, Processing Elements (PEs) (see table below).

For our experiments, we used:

- The parameters mentioned in Table 2 for the cloud.
- The following table parameters for the NSGAIII and NSGAII genetic algorithms (Table 3).

5.1. Experiment 1

In this experiment, we evaluated the results of the NSGAIII algorithm, by first comparing the results of a single run (first generation and last generation results of a run) to verify if there is an improvement of results between the first and the last generation (see Table 4).

Secondly we evaluated the results obtained by several runs with fixing the population size (100 individuals) by changing the number of generations. In Table 5 we have presented the optimal solution of each run.

NB: only one solution among the optimal solutions (1st front) will be sent and executed at the cloud level, the latter is chosen randomly.

5.2. Experiment 2

In this experiment, we made a comparative study between the algorithm proposed in this work NSGAIII besides, another Multi-objective algorithm is NSGAII, by comparing the results obtained from each algorithm (optimal solutions), as well as comparing the execution time of each algorithm. We have used the same parameters mentioned previously. The table 6 shows the results chosen and sent to the cloud as well as the run-time of the algorithms.

Table 5
Optimal solution sent to the Cloud for each generation number.

Taille de population = 100			
NB_Gen	La solution finale		
	TE (s)	coût (\$)	CE (KW)
10	18.74	0.1550897	0.1753566
100	16.098	0.1470481	0.1752485
500	15.01425	0.13514	0.1751786
1000	14.7597778	0.1303458	0.1750865
2000	14.1648889	0.1330501	0.1750424
3000	14.9874	0.1364005	0.1750021

The curves presented in Figs. 15, 16, 17 show the full optimal solutions (Pareto front) obtained for each algorithm with 1000 as generations number and 50, 200, 1000 as population number. As we have three objectives (runtime, cost, energy consumption), the curves are 3D. So that, each figure (15, 16, and 17) contains three figures representing each dimension to see closely the results.

6. Discussion and Comparative study

The first experiment aims to verify the proper functioning of the algorithm which ensures the improvement of the results between the first and the last generation on the one hand. Furthermore, to study the impact of the increasing variation in the number of generations on the size of a population determine on the other hand.

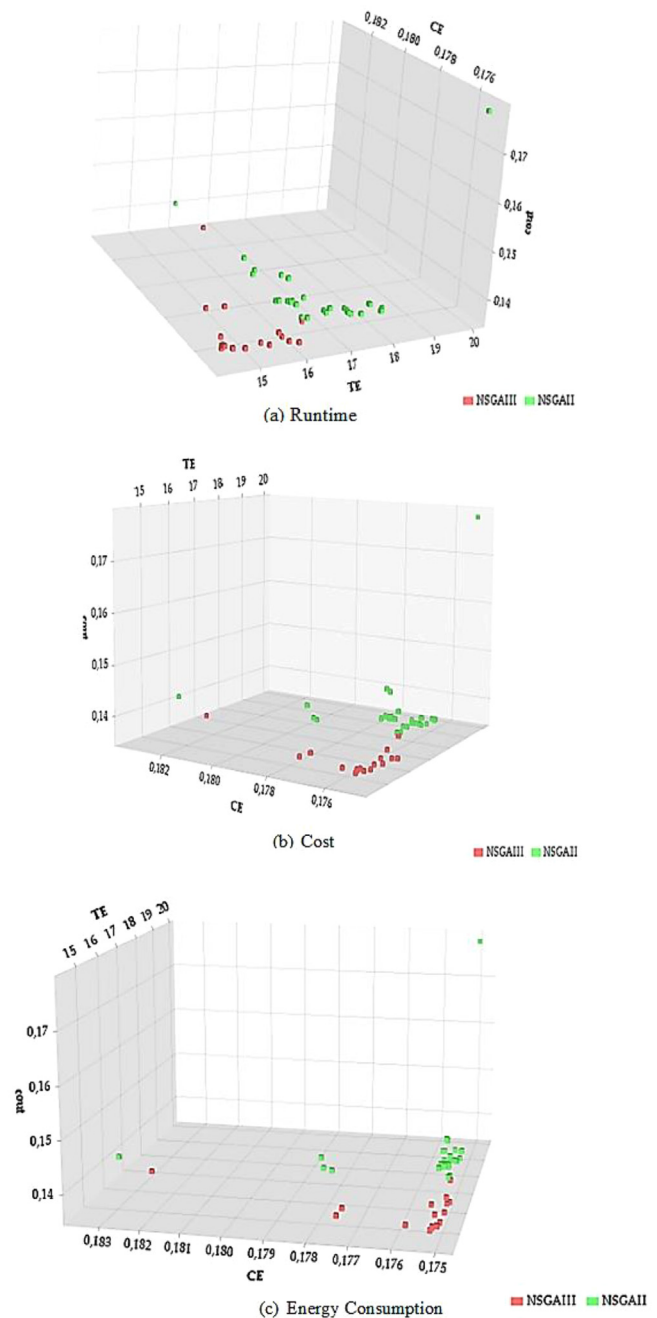


Fig. 15. NSGA-III and NSGA-II results for population size = 50.

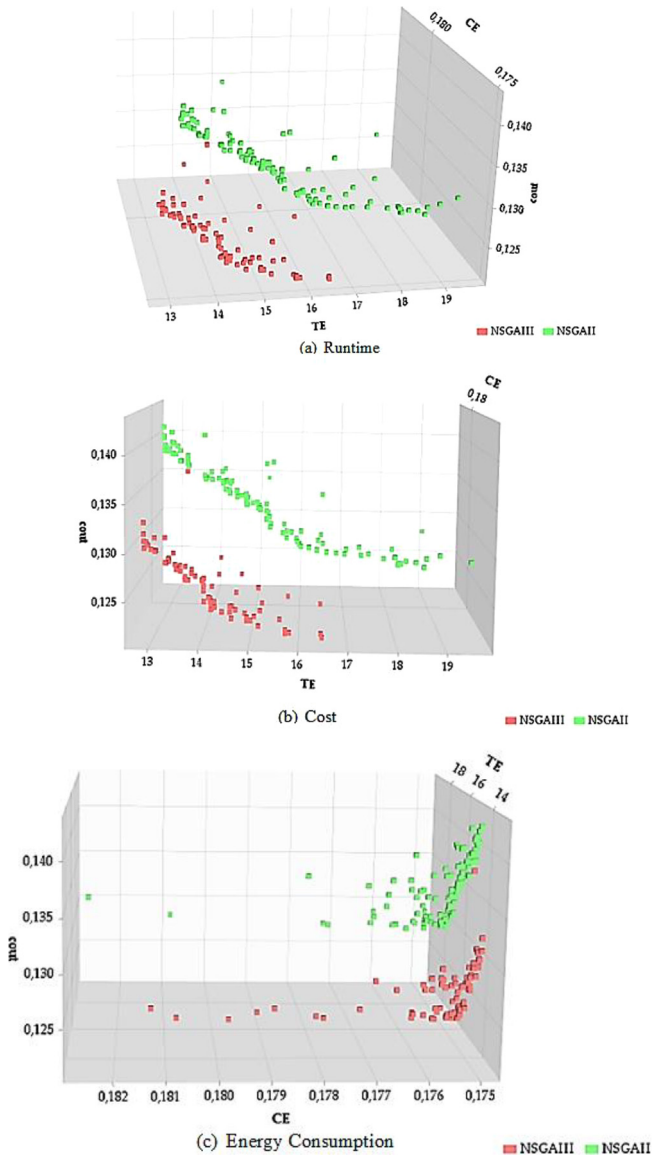


Fig. 16. NSGA-III and NSGA-II results for population size = 200.

In this experiment, we noted that the objectives values of the optimal solutions of the last generation present a clear improvement than the results of the 1st generation (1st front of the first generation). If we take for example 100 individuals, the values of the objectives are from 19 to 36 (s) for TE and from 0.17 to 0.19 (\$) for cost and from 0.17 to 0.24 Kw for energy consumption however, in the last generation the values are varied from 15 to 24 (s) for TE, from 0.14 to 0.15 (\$) for the cost and from 0.17 to 0.19 (Kw) for CE. Finally, we conclude that the algorithm does the job for which it was designed.

Table 5 presents the second part of experiment one which shows us that for a given population size the results are improved each time the number of iterations is increased, this is explained by the fact of increasing the probabilities of encountering new individuals. It also appears that after a certain number of generations any improvement disappears (number of generations = 2000). Thus, we can say that for a population size value is 100, the optimal generation is 2000.

From what precedes we conclude that the number of generations influences on the results of the algorithm.

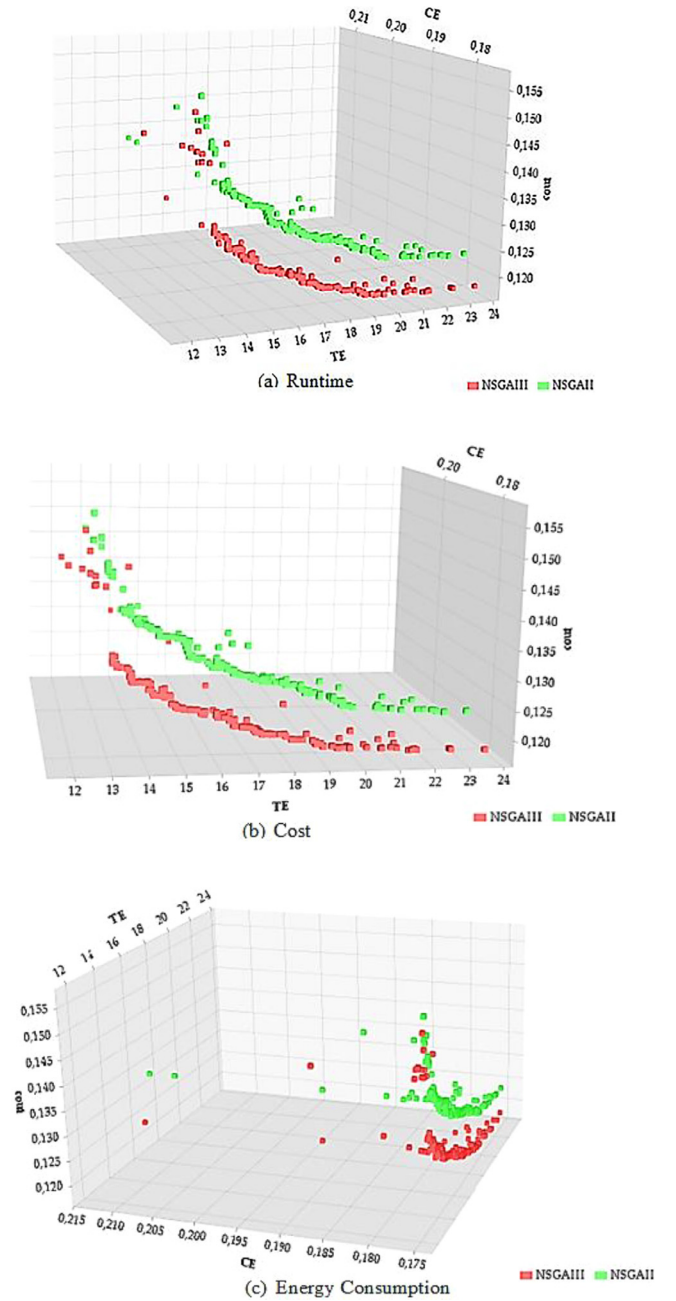


Fig. 17. NSGA-III and NSGA-II results for population size = 1000.

The second experiment aims to compare the results obtained by NSGA-III and NSGA-II with 1000 generations and the different sizes of the population.

From the results of the tables of the two experiments, we note that the NSGA-III gives better results than NSGA-II for the values of the objectives (see Table 5 and Figs. 15, 16, 17), unlike the runtime of the algorithm which is in favor of NSGA-II (see Fig. 18).

7. Complexity analysis

The efficiency measure of the algorithm allows finding a good algorithm for a given problem. Moreover, the resolution of this problem amounts to comparing the performances of two algorithms performing the same tasks. Furthermore, two essential

Table 6

Comparative study between results of the NSGAIII and NSGAI algorithms.

Number of generations = 1000					
Pop	Algo	TE(s)	Cost(\$)	CE (kw)	TE_Algo(s)
10	NSGAIII	14.832	0.1430674	0.1750402	148.05
	NSGAI	15.001	0.1427851	0.1750105	63.953
50	NSGAIII	14.55625	0.1372377	0.1750452	304.38
	NSGAI	15.84175	0.1439623	0.1752114	111.836
100	NSGAIII	14.9682	0.1312419	0.1750313	340.269 s
	NSGAI	15.4171429	0.1365634	0.1751228	199.22
200	NSGAIII	13.3874286	0.138082	0.1750111	431.566 s
	NSGAI	15.2075556	0.1344387	0.175184	297.697
500	NSGAIII	13.7374286	0.1319062	0.1750048	779.505
	NSGAI	15.2468	0.1327718	0.1750176	528.528
800	NSGAIII	12.6122	0.1327206	0.1751568	1173.0721
	NSGAI	13.957	0.145382	0.1750884	977.26
1000	NSGAIII	13.5046	0.1314738	0.1750091	1969.370
	NSGAI	15.1164	0.1365465	0.1750133	1486.762

parameters among the expensive resources are to be taken into account: the runtime (temporal complexity) and the required memory space (spatial complexity). As it permits representing the amount of memory necessary for the run of a program.

A problem is simple when the number of steps necessary for its resolution is bounded by a polynomial (polynomial problem). However, the exhaustive method is impossible to be implemented for the selection of multiple characteristics in big cloud data. This problem is characterized by a very high search space size; therefore a temporal complexity is in the worst case. The time complexity class for this problem is EXPTIME. Thus, the problem is non-polynomial (Difficult).

8. Conclusion

Cloud Computing is a new concept for deploying IT systems, it offers many advantages in terms of computing power, response

time, and cost reduction. Users can take full advantage of cloud services that meet their needs.

This work is particularly interested in the problematic scheduling of tasks in the cloud based on the selection of characteristics. The latter is considered to be a Multi-objective optimization problem, due to the conflicting nature of the QoS metrics to take into account.

Furthermore, we have adapted a metaheuristic method of Multi-objective optimization entitled the genetic algorithm of non-dominated sorting (NSGA-III) to solve this problem to optimize the runtime, the cost, as well as the energy consumption to ensure an efficient use of resources. The experiments that we carried out allowed us to verify the good functioning of NSGAIII itself on one hand and it gives better results than NSGAI for the values of objectives on the other hand. However, it should be noted that despite its effectiveness, the NSGAIII has the disadvantage of a relatively high runtime compared to the NSGAI. Moreover, as perspectives, we can cite the application of NSGAIII with more than three QoS values. The hybridization objective of NSGAIII with a metaheuristic local search algorithm at the initialization level is intended to accelerate their convergence and improve the quality of the solutions. Therefore, this hybridization is carried out at the level of final solutions to select the best of them.

Based on the work carried out, we draw up the following perspectives:

- Application of NSGA-III with more than three QoS values.
- The hybridization of NSGA-III with other Metaheuristics or local searches at the initialization level, to accelerate their convergences and improve the quality of the solutions.
- The hybridization of NSGA-III with other Metaheuristics or Local Research in terms of final solutions to improve them.
- Hybridization of NSGA-III with other Metaheuristics for intrusion detection system (IDS).
- The use of NSGAIII for big data and data mining to improve the performance of systems.

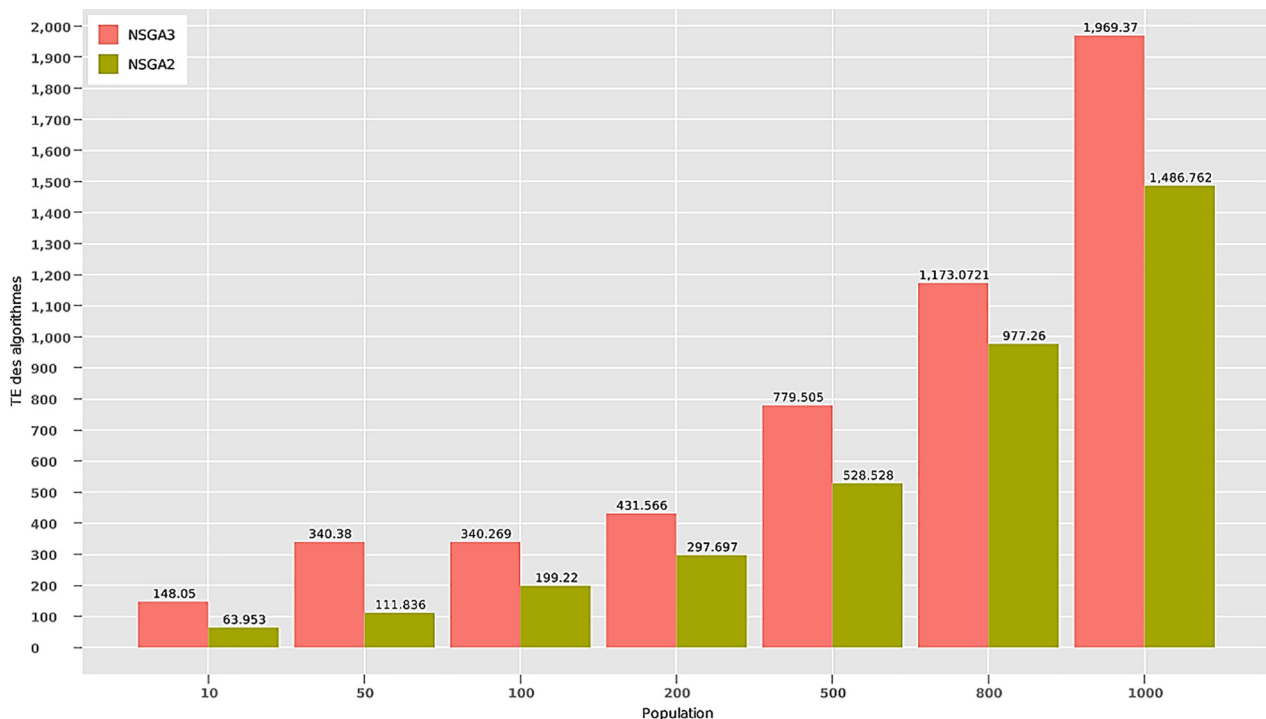


Fig. 18. Runtime Histogram of the NSGAIII and NSGAI algorithm for each run.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

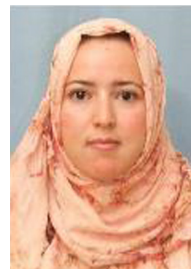
- Slah eddine, M. 2012. Cloud Computing En Afrique Situation et Perspectives, UIT.
- Thibaud, C. 2012. The challenges of Cloud Computing in company, Integration in the Cloud, Bachelor Work, Switzerland.
- Franck, B. 2007. Dimensionnement of Embedded Systems: Application to OSEK, Doctoral Thesis, National Conservatory of Arts and Crafts.
- Nassima, B. and Salima, M. 2016. Data Security in Cloud Computing, Professional Master Thesis, University A/Mira De Bejaia.
- Elmir, Y., 2017. Multi-objective optimization by genetic algorithms and Pareto approach to parameters of a PID controller. Memoire de Magister. University of Sciences and Technology of Oran Mohamed Boudiaf.
- Sonia, Y. 2014. Optimal multi-constraint allocation of workows to the resources of a Cloud Computing environment, PhD thesis.
- Insaf, D.A., 2016. Optimization of scheduling and resource allocation in Cloud Computing. University of Oran. Doctoral Thesis.
- Atul, V., Dharmendra, K.Y. 2015. Multi-objective Tasks Scheduling Algorithm for Cloud Computing Throughput Optimization, International Conference on Intelligent Computing, Communication, Convergence (ICCC), Conference Organized by Inter.
- Panda, K.I.G., Prasanta, K.J. 2015. Allocation-Aware Task Scheduling for Heterogeneous Multi-Cloud Systems, 2nd International Symposium on Big Data and Cloud Computing (ISBCC'15), Veer Surendra Sai University of Technology, Burla – 768018, India.
- Jena, R.K. 2015. Multi Objective Task Scheduling In Cloud Environment Using Nested Pso Framework, Institute Of Management Technology, Elsevier.
- Shekhar, S.M.K. 2014. Task Scheduling Optimization Of Independent Tasks In Cloud Computing Using Enhanced Genetic Algorithm, Department of Computer Science, Nitttr, Panjab University, Chandigarh.
- Slim, B., Rituparna, D.A.G. 2016. Recent Advances in Evolutionary Multi-objective Optimization, Edition Springer International Publishing AG, pages 125, 126.
- Kalyanmoy, D., Fellow, H.J., 2013. An Evolutionary Many- Objective Optimization Algorithm Using Reference-point Based Non-dominated Sorting Approach, Part I: Solving Problems with Box Constraints. IEEE.
- Haitham, S., Kalyanmoy, D., 2017. U-NSGA-III: A Unified Evolutionary Algorithm for Single, Multiple, and Many-Objective Optimization. Computational Optimization and Innovation Laboratory (COIN). Department of Computer Science and Engineering. Michigan State University, USA.
- Mahdi, S., 2007. Multi-objective Optimization Using A Novel Scheme Of Meta/Exact Cooperation. University of Mentouri Constantine. Magister Thesis.
- Layeb, A., 2010. Utilisation des Approches d'Optimisation Combinatoire pour la Vérification des Applications Temps Réel. Université Mentouri de Constantine, Thèse de Doctorat.
- El-Ghazali Talbi. (Décembre 2000). Méta heuristiques pour l'Optimisation Combinatoire multi objectifs: Etat de l'art. Article. université de Lille Nord de France.
- Yuan, Y., Xu, H., Wang, B. (2014, July). An improved NSGA-III procedure for evolutionary many-objective optimization. In Proceedings of the 2014 annual conference on genetic and evolutionary computation (pp. 661-668).
- Ibrahim, A., Rahnamayan, S., Martin, M.V., Deb, K. (2016, July). EliteNSGA-III: An improved evolutionary many-objective optimization algorithm. In 2016 IEEE Congress on evolutionary computation (CEC) (pp. 973-982). IEEE.
- Zuo, L., Shu, L., Dong, S., Zhu, C., Hara, T., 2015. A Multi-objective optimization scheduling method based on the ant colony algorithm in cloud computing. IEEE Access 3, 2687–2699.
- Naik, K., Gandhi, G.M., & Patil, S.H. (2019). Multi-objective virtual machine selection for task scheduling in cloud computing. In Computational Intelligence: theories, applications and future directions-volume I (pp. 319-331). Springer, Singapore.
- Adamuthe, A.C., Pandharpatte, R.M., & Thampi, G.T. (2013, November). Multi-objective virtual machine placement in cloud environment. In 2013 International Conference on Cloud & Ubiquitous Computing & Emerging Technologies (pp. 8-13). IEEE.
- Zhou, Z., Li, F., Zhu, H., Xie, H., Abawajy, J.H., Chowdhury, M.U., 2020. An improved genetic algorithm using greedy strategy toward task scheduling optimization in cloud environments. Neural Comput. Appl. 32 (6), 1531–1541.
- Alkayal, E.S., Jennings, N.R., & Abulkhair, M.F. (2016, November). Efficient task scheduling Multi-objective particle swarm optimization in cloud computing. In

2016 IEEE 41st Conference on Local Computer Networks Workshops (LCN Workshops) (pp. 17-24). IEEE.

Yuan, Y., Xu, H., Wang, B., 2015. July). An experimental investigation of variation operators in reference-point based many-objective optimization. In: In Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation, pp. 775–782.



Latreche Imene a PhD student at Computer sciences department in Mohamed khider university, Biskra, Algeria. She got her master diploma in 2018 (option: Artificial intelligence), on the work: "multi-objective optimization for tasks scheduling on cloud computing". Her areas of specializations are Artificial intelligence and optimization problems.



Dr. Sihem Slatnia was born in the city of Biskra, Algeria. She followed her high studies at the university of Biskra, Algeria at the Computer Science Department and obtained the engineering diploma in 2004 on the work; "Diagnostic based model by Black and White analyzing in Background Petri Nets". After that, she obtained Master diploma in 2007 (option: Artificial intelligence and advanced system's information), on the work: "Evolutionary Cellular Automata Based-Approach for Edge Detection". She obtained PhD degree from the same university in 2011, on the work "Evolutionary Algorithms for Image Segmentation based on Cellular Automata". Presently she is an associate professor at computer science department of Biskra University. She is interested to the artificial intelligence, emergent complex systems and optimization.



Prof. Kazar Okba Obtained his Magister degree by 1997 from Constantine University (Algeria). He obtained his PhD degree from the same university by 2005. He is member of editorial board of some international journal and author of some publication in international journals. Okba participates as a member of program committee in international conferences. He is interested and working in artificial intelligence field and multi-agents systems with their applications and also advanced information systems, web services, semantic web. He actually is a Full Professor at Computer Science Department of Biskra University. He is visiting professor at the United Arab Emirate University.



Prof. Batouche Mohamed Mohamed Batouche, PhD, is currently a full Professor at Princess Nourah University, Riyadh, Saudi Arabia. He was also full Professor and head of the department of Computer Science at University of Constantine, Algeria. He received Engineer degree in Computer Science at University of Constantine, Algeria, MSc and PhD degrees in Computer Science at University of Nancy - INPL, France. He published over 250 refereed journals and conference papers in the areas of Complex Systems, Bioinformatics, Machine Learning, Deep Learning, Cloud Computing, Big Data, and Cyber Security.